

PipeSD 库调用与新范式扩展指南

?????? ? ????? ? Agent ????

?????? 2?5 ????????? 6?8 ??????????? 9?18 ??? Agent ???????????????????????????

面向：希望调用 PipeSD 完成任务的使用者，以及希望接入 Routing、Cascade、Agent、Split Inference 等新端云协同范式的开发者。

1. 先理解“调用”和“扩展”的区别

调用现有能力：调用者不修改 PipeSD 核心代码，只准备模型、输入和 Config，然后执行文本或视频 Engine。

扩展新范式：开发者增加新的 Backend/Node、Strategy 和 Engine，复用现有 Task、Result、Channel、配置和指标体系。

不建议复制 `run_edge.py` 再修改成一个孤立脚本。正确方式是把变化放在 Strategy 和专用 Engine 中，把模型差异放在 Backend/Node 中。

2. 调用方式一：统一命令行

文本：

```
python -m edge.app.run text --config configs/text-local.json
```

视频：

```
python -m edge.app.run video --config configs/video-local.json
```

命令行可以覆盖 Config：

```
python -m edge.app.run text \
  --config configs/text-local.json \
  --chunk-size 8 \
  --max-new-tokens 128
```

适合场景：实验运行、批量评测、服务器部署、快速复现。

3. 调用方式二：Python SDK

现有 Engine 都遵循统一的 `run(Task) -> Result`：

```
from pipesd import Task

task = Task(
    task_id="video-001",
    modality="video",
    input_data="/data/example.mp4",
    prompt="Please describe the video in detail.",
    metadata={"dataset": "vdc"},
)

result = engine.run(task)
print(result.output)
print(result.metrics)
```

调用者需要提供：

- ? Task: 任务编号、模态、输入、Prompt 和可选 metadata。
- ? Engine: 已经装配好 Edge Node、Cloud Node、Strategy 和 Channel 的运行引擎。
- ? Config: 模型路径、服务地址、算法参数和输出位置。

调用者得到 Result:

- ? **task_id**: 任务编号。
- ? **output**: 文本、分类结果、结构化答案等。
- ? **status**: completed、failed 或 cancelled。
- ? **stop_reason**: EOS、最大长度、策略终止等。
- ? **metrics**: 耗时、吞吐量、云查询次数、通信量等。
- ? **metadata**: token、路由轨迹或任务附加信息。

4. 调用方式三：包装现有 Python 模型

已有模型对象不一定要继承 Node。可以先用 BackendNode 包装：

```
from pipesd import BackendNode, NodeCapabilities

edge = BackendNode(
    backend=my_local_model,
    node_id="edge-small-model",
    location="edge",
    capabilities=NodeCapabilities(
        modalities=("text",),
        models=("my-model",),
        supports_logits=True,
    ),
)

output = edge.invoke("generate", "hello", max_new_tokens=32)
```

只要 Backend 提供 Engine 所需要的方法，就可以先适配，不必重写模型内部推理代码。

5. 调用方式四：远程 Cloud Node

Cloud 通过 HTTPNode 表示：

```
from pipesd import HTTPNode

cloud = HTTPNode(
    "http://cloud-host:8000",
    channel,
    node_id="cloud-large-model",
    endpoints={
        "init": "/init",
        "propose": "/propose",
        "exit": "/exit",
    },
)
```

```
    },  
    )  
  
    health = cloud.health()
```

Engine 只面向 Node 操作，不需要知道 Cloud 使用 FastAPI、vLLM、Ollama、普通云 API 还是未来的 SPEAR 节点。

6. 新范式扩展的标准步骤

建议按以下顺序增加任何新范式：

1. 明确 Task 输入、Result 输出和端云交互状态。
2. 将已有模型实现包装为 BackendNode，或实现新的 Node。
3. 定义 Strategy，只负责决策，不负责完整调度循环。
4. 定义专用 Engine，执行该范式的生命周期。
5. 复用现有 Channel；只有传输需求不同才增加新 Channel。
6. 增加 Config section 和任务注册项。
7. 先用 InProcessChannel 和 Mock Node 测试。
8. 再用 HTTP Channel 和真实模型做回归。
9. 统一输出 Result 和指标，不另建私有结果格式。

7. Strategy 模板

```
from pipesd import Action, Decision, Strategy  
  
class ConfidenceRoutingStrategy(Strategy):  
    def __init__(self, threshold=0.8):  
        self.threshold = threshold  
  
    def decide(self, context):  
        confidence = context.observations["confidence"]  
        if confidence >= self.threshold:  
            return Decision(  
                action=Action.ACCEPT_LOCAL,  
                target="edge",  
                reason="edge confidence is sufficient",  
            )  
        return Decision(  
            action=Action.RUN_CLOUD,  
            target="cloud",  
            reason="edge confidence is below threshold",  
        )
```

Strategy 输入 CollaborationContext：

- ? **task**: 当前任务。
- ? **state**: Engine 的可变协同状态。
- ? **observations**: 模型置信度、风险分数、预算等观测。

? **metrics**: 累计延迟、费用、带宽等指标。

Strategy 输出 Decision: Action、target、reason、payload 和 metadata。

8. Engine 模板

```
from pipesd import CollaborationContext, Engine, Result

class RoutingEngine(Engine):
    def __init__(self, edge_node, cloud_node, strategy):
        self.edge = edge_node
        self.cloud = cloud_node
        self.strategy = strategy

    def run(self, task):
        edge_result = self.edge.invoke("infer", task.input_data, task.prompt)
        decision = self.strategy.decide(CollaborationContext(
            task=task,
            observations={"confidence": edge_result["confidence"]},
        ))

        if decision.target == "edge":
            output = edge_result["output"]
            route = "edge"
        else:
            cloud_result = self.cloud.invoke("infer", task)
            output = cloud_result["output"]
            route = "cloud"

        return Result(
            task_id=task.task_id,
            output=output,
            metrics={"route": route},
            metadata={"decision_reason": decision.reason},
        )
```

Engine 负责循环、状态、错误处理和结果聚合；Strategy 只决定下一步动作。

9. Agent 端云协同应该怎样设计

Agent 与推测解码不同：协同单位不再只是 token，而可能是用户请求、对话轮次、计划步骤或工具调用。

推荐流程：

```
用户请求
↓
Edge Agent: 意图识别、隐私检测、简单推理
↓
Agent Strategy: 本地完成 / 脱敏上云 / 拒绝上云 / Cloud 推理
↓
Cloud Agent: 复杂规划、大模型推理、远程工具
↓
Tool Policy: 检查工具权限和副作用
↓
Edge 汇总、恢复本地上下文、返回 Result
```

10. Agent Task 与 Result

Agent Task 可以使用现有 Task:

```
task = Task(
    task_id="agent-001",
    modality="agent",
    input_data={
        "messages": [{"role": "user", "content": "帮我分析本地文档"}],
        "available_tools": ["local_search", "cloud_reasoner"],
    },
    metadata={
        "privacy_level": "private",
        "latency_budget_ms": 3000,
        "cloud_budget": 1,
    },
)
```

Agent Result 可以包含最终回答、路由轨迹和工具调用记录:

```
Result(
    task_id="agent-001",
    output="最终回答",
    metrics={
        "cloud_calls": 1,
        "tool_calls": 2,
        "total_time_s": 2.4,
    },
    metadata={
        "route_trace": ["edge", "redact", "cloud", "edge"],
        "tool_trace": [],
    },
)
```

11. Agent Node 划分

建议至少定义以下节点:

- ? EdgeModelNode: 本地小模型或本地 Agent runtime。
- ? CloudModelNode: 远程大模型或远程 Agent runtime。
- ? LocalToolNode: 本地文件、传感器和私有数据库工具。
- ? CloudToolNode: 搜索、远程数据库和云端服务。
- ? PolicyNode: 可选的权限、隐私和安全检查节点。

工具也是计算单元, 因此可以按 Node 统一管理, 不必全部塞进 AgentEngine。

12. Agent Strategy 示例

```
from pipesd import Action, Decision, Strategy

class PrivacyAwareAgentStrategy(Strategy):
    def decide(self, context):
        risk = context.observations.get("privacy_risk", 0.0)
```

```

difficulty = context.observations.get("difficulty", 0.0)
cloud_calls = context.metrics.get("cloud_calls", 0)

if risk >= 0.8:
    return Decision(Action.REDACT, target="edge",
                    reason="sensitive content must be redacted")
if difficulty < 0.5:
    return Decision(Action.RUN_EDGE, target="edge",
                    reason="edge model is sufficient")
if cloud_calls >= 1:
    return Decision(Action.FALLBACK, target="edge",
                    reason="cloud budget exhausted")
return Decision(Action.RUN_CLOUD, target="cloud",
                reason="complex request needs cloud reasoning")

```

实际 Agent Strategy 还可以考虑网络状态、费用、工具权限、电量、用户偏好和数据驻留要求。

13. AgentEngine 生命周期建议

```

initialize(Task)
→ Edge 理解输入
→ Strategy.decide(Context)
→ 可选脱敏
→ Edge/Cloud 模型执行
→ 可选工具调用与权限检查
→ 更新 Context
→ 循环直到 STOP
→ Result
→ finalize(task_id)

```

必须设置最大步骤数、Cloud 调用预算、工具超时和取消机制，避免 Agent 无限循环。

14. Agent 协议建议

不要直接把推测 token 的 `/propose` 字段强行用于 Agent。可以复用生命周期思想，但定义独立资源：

```

POST /v1/agent/init
POST /v1/agent/step
POST /v1/agent/tool-result
POST /v1/agent/exit
GET /health

```

每个请求至少携带：protocol_version、task_id、request_id、sequence_no、revision 和 payload。Cloud 必须维护多任务隔离、幂等性、超时清理和结构化错误响应。

15. Channel 何时需要扩展

普通请求/响应继续使用 NetworkChannel。以下场景再增加 Channel：

- ？ Agent 流式输出：SSE 或 WebSocket Channel。
- ？ 高频二进制中间结果：gRPC Channel。
- ？ 单元测试或嵌入式调用：InProcessChannel。

- ? 接入 SPEAR: SpearChannel 或 SpearNode Adapter。
- ? 接入 EdgeClaw/Ollama/vLLM: 优先实现 Node Adapter, 协议特殊时再实现 Channel。

16. 新范式的推荐目录

```
pipesd/
├── engines/agent.py
├── strategies/agent.py
└── nodes/agent.py

edge/
├── families/agent/
│   ├── backends.py
│   ├── agent_edge.py
│   └── config.py
└── app/run.py          # 注册 agent modality

cloud/
├── core/agent_tasks.py
├── models/agent_target.py
└── app/main.py        # 注册 /v1/agent/*

shared/
└── agent_protocol.py

configs/
└── agent.example.json
```

17. Agent Config 示例

```
{
  "common": {
    "server_url": "http://127.0.0.1:8000",
    "server_timeout_s": 120,
    "output_jsonl": "edge/exp/results/agent_results.jsonl"
  },
  "agent": {
    "edge_backend": "ollama",
    "edge_model": "qwen-small",
    "cloud_backend": "vllm",
    "cloud_model": "qwen-large",
    "strategy": "privacy_aware",
    "max_steps": 8,
    "max_cloud_calls": 2,
    "privacy_threshold": 0.8,
    "allowed_tools": ["local_search", "web_search"]
  }
}
```

最终调用应保持统一：

```
python -m edge.app.run agent --config configs/agent-local.json
```

18. 测试要求

每个新范式至少包含：

- ? Strategy 决策边界测试。
- ? Engine Task → Result 测试。
- ? InProcessChannel 端到端协议测试。
- ? HTTP Mock Cloud 测试。
- ? 超时、重复 request_id、版本错误和 Cloud 失败测试。
- ? 多任务状态隔离与任务清理测试。
- ? 真实模型小样本回归。
- ? 指标字段、输出 JSONL 和旧入口兼容测试。

Agent 还必须测试隐私不出端、工具权限、Cloud 预算、最大步骤数和取消行为。

19. 不建议的做法

- ? 不要在一个进程里直接加载 Edge 和 Cloud 模型来绕过协议。
- ? 不要把所有范式塞进一个巨大 Engine 循环。
- ? 不要让 Strategy 直接执行网络请求或写结果文件。
- ? 不要让 Engine 依赖具体 GPU 编号或具体模型类。
- ? 不要让 Agent 未经策略检查直接调用具有副作用的工具。
- ? 不要为每个范式重新发明 Task、Result、错误格式和指标格式。

20. 接入验收清单

1. 新范式可以通过统一 Task 输入和 Result 输出。
2. Engine 不直接依赖模型部署位置。
3. 决策逻辑集中在 Strategy。
4. 本地和远端模型通过 Node 表示。
5. 网络通信通过 Channel 完成。
6. Cloud 状态按 task_id 隔离并能清理。
7. Config 可保存并复现实验。
8. Mock 与真实模型测试均通过。
9. 指标能说明计算、通信、路由和任务效果。
10. 旧文本和视频范式不受影响。